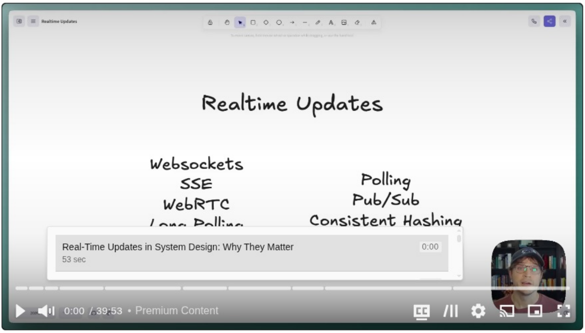
Limited Time Offer: **Up to 25% off select mock interviews** 🎉

Patterns

Real-time Updates

Learn about methods for triggering real-time updates in your system design

Real-time Updates



⚡ **Real-time Updates** addresses the challenge of delivering immediate notifications and data changes from servers to clients as events occur. From chat applications where messages need instant delivery to live dashboards showing real-time metrics, users expect to be notified the moment something happens. This pattern covers the architectural approaches to enable low-latency, bidirectional communication.

The Problem

Consider a collaborative document editor like Google Docs. When one user types a character, all other users viewing the document need to see that change within milliseconds. In apps like this you can't have every user constantly polling the server for updates every few milliseconds without crushing your infrastructure.

The core challenge is establishing efficient, persistent communication channels between clients and servers. Standard HTTP follows a request-response model: clients ask for data, servers respond, then the connection closes. This works great for traditional web browsing but breaks down when you need servers to proactively push updates to clients.

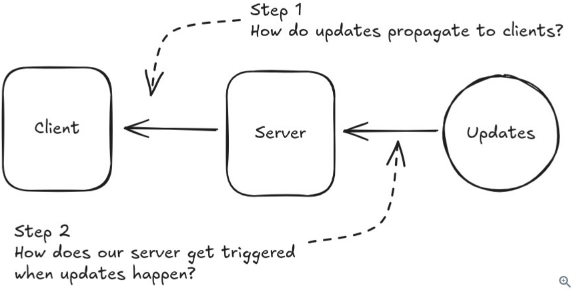
Unfortunately for many candidates, these problems (if they are faced at all) are often solved once by a specialized team. This means that many design challenges will require you to cross a bridge you may not have had the opportunity to build (I've spoken with dozens of candidates with 10+ years of experience for whom this was the case). Don't worry though, in this pattern we'll cover the important things you know to be able to make great decisions in your interview. And who knows, maybe one day you'll be the one building these pieces of your next project!

The Solution

When systems require real-time updates, push notifications, etc, the solution requires two distinct pieces:

1. The first "hop": how do we get updates from the server to the client?
2. The second "hop": how do we get updates from the source to the server?

Realtime Updates


Step 1
How do updates propagate to clients?
Client
Server
Updates
Step 2
How does our server get triggered
when updates happen?

Two Hops for Real-time Updates

We'll break down each hop separately as they involve different trade-offs which work together.

Client-Server Connection Protocols

The first "hop" is establishing efficient communication channels between clients and servers. While traditional HTTP request-response works for a startling number of use-cases, real-time systems frequently need persistent connections or clever polling strategies to enable servers to **push** updates to clients. This is where we get into the nitty-gritty of networking.

Networking 101

Before diving into the different protocols for facilitating real-time updates, it's helpful to understand a bit about how networking works — in some sense the problems we're talking about here are just networking problems! Networks are built on a layered architecture (the so-called **"OSI model"**) which greatly simplifies the world for us application developers who sit on top of it.

Networking Layers

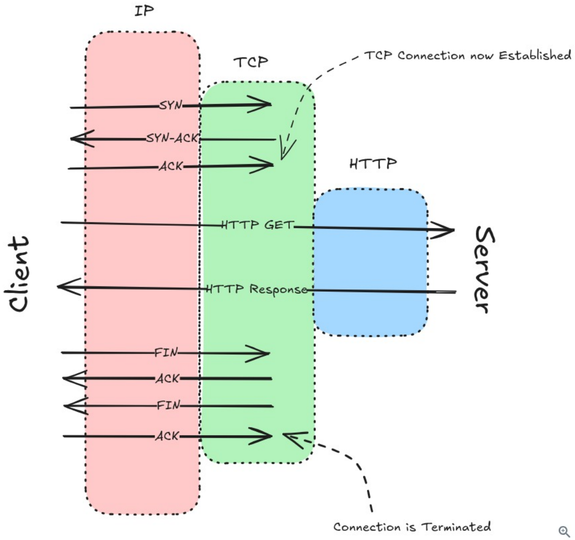
In networks, each layer builds on the **abstractions** of the previous one. This way, when you're requesting a webpage, you don't need to know which voltages represent a `1` or a `0` on the network wire - you just need to know how to use the next layer down the stack. While the full networking stack is fascinating, there are three key layers that come up most often in system design interviews:

- **Network Layer (Layer 3):** At this layer is IP, the protocol that handles routing and addressing. It's responsible for breaking the data into packets, handling packet forwarding between networks, and providing best-effort delivery to any destination IP address on the network. However, there are no guarantees: packets can get lost, duplicated, or reordered along the way.
- **Transport Layer (Layer 4):** At this layer, we have TCP and UDP, which provide end-to-end communication services:
  - TCP is a **connection-oriented** protocol: before you can send data, you need to establish a connection with the other side. Once the connection is established, it ensures that the data is delivered correctly and in order. This is a great guarantee to have but it also means that TCP connections take time to establish, resources to maintain, and bandwidth to use.
  - UDP is a **connectionless** protocol: you can send data to any other IP address on the network without any prior setup. It does not ensure that the data is delivered correctly or in order. Spray and pray!
- **Application Layer (Layer 7):** At the final layer are the application protocols like DNS, HTTP, Websockets, WebRTC. These are common protocols that build on top of TCP to provide a layer of abstraction for different types of data typically associated with web applications. We'll get into them in a bit!

These layers work together to enable all our network communications. To see how they interact in practice, let's walk through a concrete example of how a simple web request works.

Request Lifecycle

When you type a URL into your browser, several layers of networking protocols spring into action. Let's break down how these layers work together to retrieve a simple web page over HTTP. First, we use DNS to convert a human-readable domain name like `hellointerview.com` into an IP address like `32.42.52.62`. Then, a series of carefully orchestrated steps begins:



Simple HTTP Request

1. **DNS Resolution**: The client starts by resolving the domain name of the website to an IP address using DNS (Domain Name System)[1].
2. **TCP Handshake**: The client initiates a TCP connection with the server using a three-way handshake:
    - **SYN**: The client sends a SYN (synchronize) packet to the server to request a connection.
    - **SYN-ACK**: The server responds with a SYN-ACK (synchronize-acknowledge) packet to acknowledge the request.
    - **ACK**: The client sends an ACK (acknowledge) packet to establish the connection.
3. **HTTP Request**: Once the TCP connection is established, the client sends an HTTP GET request to the server to request the web page.
4. **Server Processing**: The server processes the request, retrieves the requested web page, and prepares an HTTP response.
5. **HTTP Response**: The server sends the HTTP response back to the client, which includes the requested web page content.
6. **TCP Teardown**: After the data transfer is complete, the client and server close the TCP connection using a four-way handshake:
    - **FIN**: The client sends a FIN (finish) packet to the server to terminate the connection.
    - **ACK**: The server acknowledges the FIN packet with an ACK.
    - **FIN**: The server sends a FIN packet to the client to terminate its side of the connection.
    - **ACK**: The client acknowledges the server's FIN packet with an ACK.

While the specific details of TCP handshakes might seem technical, two key points are particularly relevant for system design interviews:

1. First, each round trip between client and server adds **latency** to our request, including those to establish connections before we send our application data.

2. Second, the TCP connection itself represents **state** that both the client and server must maintain. Unless we use features like HTTP keep-alive, we need to repeat this connection setup process for every request - a potentially significant overhead.

Understanding when connections are established and how they are managed is crucial to touching on the important choices relevant for realtime updates.

> It's less common recently in BigTech, but it used to be a popular interview question to ask candidates to dive into the details of "what happens when you type (e.g.) `hellointerview.com` into your browser and press enter?".
>
> Details like these aren't typically a part of a system design interview but it's helpful to understand the basics of networking. It may save you some headaches on the job!

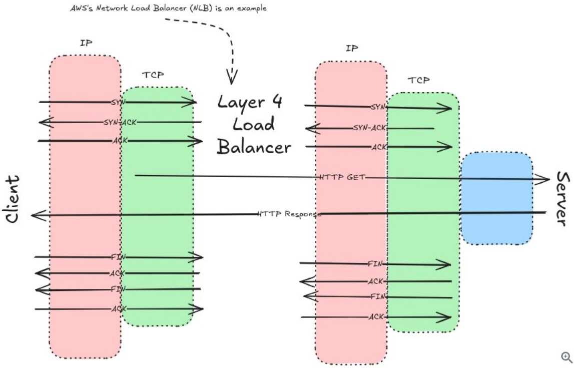
With Load Balancers

In real-world systems, we typically have multiple servers working together behind a load balancer. Load balancers distribute incoming requests across these servers to ensure even load distribution and high availability. There are two main types of load balancers you'll encounter in system design interviews: Layer 4 and Layer 7.

These will have some implications for decisions we'll make later, but let's briefly cover the difference between the two.

Layer 4 Load Balancers

Layer 4 load balancers operate at the transport layer (TCP/UDP). They make routing decisions based on network information like IP addresses and ports, without looking at the actual content of the packets. The effect of a L4 load balancer is as-if you randomly selected a backend server and assumed that TCP connections were established directly between the client and that server — this mental model is not far off.



Simple HTTP Request with L4 Load Balancer

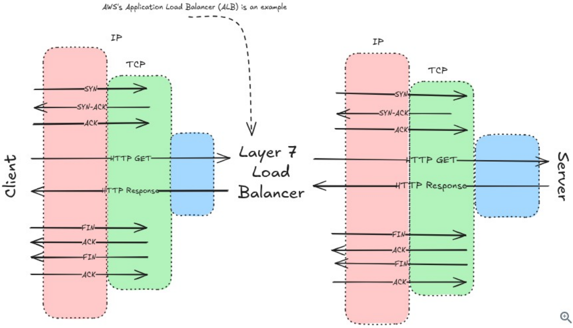
Key characteristics of L4 load balancers:

- Maintain persistent TCP connections between client and server.
- Fast and efficient due to minimal packet inspection.
- Cannot make routing decisions based on application data.
- Typically used when raw performance is the priority.

For example, if a client establishes a TCP connection through an L4 load balancer, that same server will handle all subsequent requests within that TCP session. This makes L4 load balancers particularly well-suited for protocols that require persistent connections, like WebSocket connections. At a conceptual level, *it's as if we have a direct TCP connection between client and server which we can use to communicate at higher layers.*

Layer 7 Load Balancers

Layer 7 load balancers operate at the application layer, understanding protocols like HTTP. They can examine the actual content of each request and make more intelligent routing decisions.



Simple HTTP Request with L7 Load Balancer

Key characteristics of L7 load balancers:

- Terminate incoming connections and create new ones to backend servers.
- Can route based on request content (URL, headers, cookies, etc.).
- More CPU-intensive due to packet inspection.
- Provide more flexibility and features.
- Better suited for HTTP-based traffic.

For example, an L7 load balancer could route all API requests to one set of servers while sending web page requests to another (providing similar functionality to an **API Gateway**), or it could ensure that all requests from a specific user go to the same server based on a cookie. The underlying TCP connection that's made to your server via an L7 load balancer is not all that relevant! It's just a way for the load balancer to forward L7 requests, like HTTP, to your server.

While L7 load balancers can help us to not have to worry about lower-level details like TCP connections, we aren't able to ignore the connection-level reality if we want persistent connections to consistent servers.

> The choice between L4 and L7 load balancers often comes up in system design interviews when discussing real-time features. There are some L7 load balancers which explicitly support connection-oriented protocols like WebSockets, but generally speaking L4 load balancers are better for WebSocket connections, while L7 load balancers offer more flexibility for HTTP-based solutions like long polling. We'll get into more detail on this in the next section.

Alright, now that we covered the necessary networking concepts, let's dive into the different approaches for facilitating real-time updates between clients and servers, our first "hop". As a motivating example, let's consider a chat application where users need to receive new messages sent to the chat room they are a part of.

Simple Polling: The Baseline

The simplest possible approach is to have the client regularly poll the server for updates. This could be done with a simple HTTP request that the client makes at a regular interval. This doesn't technically qualify as real-time, but it's a good starting point and provides a good contrast for our other methods.

> A lot of interview questions don't *actually* require real-time updates. Think critically about the product and ask yourself whether lower frequency updates (e.g. every 2-5 seconds) would work. If so, you may want to propose a simple, polling-based approach. It's better to suggest a less-than-perfect solution than to fail to implement a complex one.
>
> That said, do make this proposal to your interviewer before pulling the trigger. If they are dead-set on you talking about WebSockets, SSE, or WebRTC, you'll want to know that sooner than later!

How does it work? It's dead simple! The client makes a request to the server at a regular interval and the server responds with the current state of the world. In our chat app, we would just constantly be polling for "what messages have I not received yet?".

```
async function poll() {
  const response = await fetch('/api/updates');
  const data = await response.json();
```

```
  processData(data);
}

// Poll every 2 seconds
setInterval(poll, 2000);
```

Advantages

- Simple to implement.
- Stateless.
- No special infrastructure needed.
- Works with any standard networking infrastructure.
- Doesn't take much time to explain.

> This last point is underrated. If the crux of your problem is *not* real-time updates, you'll want to propose a simple, polling-based approach. You'll preserve your mental energy and interview time for the parts of the system that truly matter.

Disadvantages

- Higher latency than other methods. Updates might be delayed as long as the polling interval + the time it takes to process the request.
- Limited update frequency.
- More bandwidth usage.
- Can be resource-intensive with many clients, establishing new connections, etc.

When to use simple polling

Simple polling is a great baseline and, absent a problem which specifically requires very-low latency, real-time updates, it's a great solution. It's also appropriate when the window where you need updates is short, like in our **Leetcode system design**.

Things to Discuss in Your Interview

You'll want to be clear with your interviewer about the trade-offs you're making with polling vs other methods. A good explanation highlights the simplicity of the approach and gives yourself a backdoor if you discover that you need something more sophisticated. "I'm going to start with a simple polling approach so I can focus on X, but we can switch to something more sophisticated if we need to later."

The most common objection from interviewers to polling is either that it's too slow or that it's not efficient. Be prepared to discuss why the polling frequency you've chosen is appropriate and sufficient for the problem. On the efficiency front, it's great to be able to discuss how you can reduce the overhead. One way to do this is to take advantage of HTTP keep-alive connections. Setting an HTTP keep-alive which is longer than the polling interval will mean that, in most cases, you'll only need to establish a TCP connection once which minimizes some of the setup and teardown overhead.

Long Polling: The Easy Solution

After a baseline for simple polling, long polling is the easiest approach to achieving near real-time updates. It builds on standard HTTP, making it easy to implement and scale.

The idea is also simple: the client makes a request to the server and the server holds the request open until new data is available. It's as if the server is just taking really long to process the request. The server then responds with the data, finalizes the HTTP requests, and the client immediately makes a **new** HTTP request. This repeats until the server has new data to send. If no data has come through in a long while, we might even return an empty response to the client so that they can make another request.

For our chat app, we would keep making a request to get the *next message*. If there was no message to retrieve, the server would just hold the request open until a new message was sent before responding to us. After we received that message, we'd make a new request for the next message.

1. Client makes HTTP request to server
2. Server holds request open until new data is available
3. Server responds with data
4. Client immediately makes new request

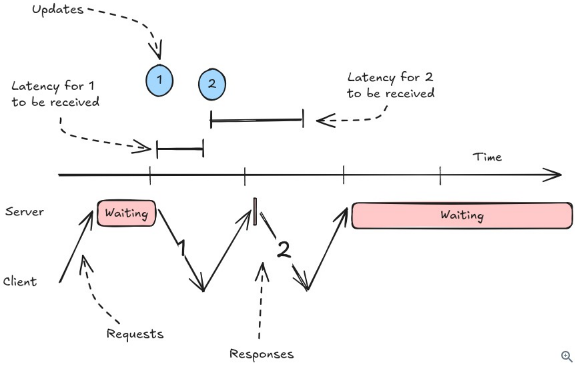
5. Process repeats

```
// Client-side of long polling
async function longPoll() {
  while (true) {
    try {
      const response = await fetch('/api/updates');
      const data = await response.json();

      // Handle data
      processData(data);
    } catch (error) {
      // Handle error
      console.error(error);

      // Add small delay before retrying on error
      await new Promise(resolve => setTimeout(resolve, 1000));
    }
  }
}
```

The simplicity of the approach hides an important trade-off for high-frequency updates. Since the client needs to "call back" to the server after each receipt, the approach can introduce some extra latency:



Long Polling Latency

> *Assume the latency between the client and server is 100ms.*
>
> *If we have 2 updates which occur within 10ms of each other, with long polling we'll receive the first update 100ms after it occurred (100ms of network latency) but the second update may be up to 290ms after it occurred (90ms for the first response to finish returning, 100ms for the second request to be made, and another 100ms to get the response).*

Advantages

- Builds on standard HTTP and works everywhere HTTP works.
- Easy to implement.
- No special infrastructure needed.
- Stateless server-side.

Disadvantages

- Higher latency than alternatives.
- More HTTP overhead.
- Can be resource-intensive with many clients.
- Not suitable for frequent updates due to the issues mentioned above.

- Makes monitoring more painful since requests can hang around for a long time.
- Browsers limit the number of concurrent connections per domain, meaning you may only be able to have a few long-polling connections per domain.

When to Use Long Polling

Long polling is a great solution for near real-time updates with a simple implementation. It's a good choice when updates are infrequent and a simple solution is preferred. If the latency trade-off of a simple polling solution is at all an issue, long-polling is an obvious upgrade with minimal additional complexity.

Long Polling is a great solution for applications where a long async process is running but you want to know when it finishes, as soon as it finishes - like is often the case in payment processing. We'll long-poll for the payment status before showing the user a success page.

Things to Discuss in Your Interview

Because long-polling utilizes the existing HTTP infrastructure, there's not a bunch of extra infrastructure you're going to need to talk through. Even though the polling is "long", you still do need to be specific about the polling frequency. Keep in mind that each hop in your infrastructure needs to be aware of these lengthy requests: you don't want your load balancer hanging up on the client after 30 seconds when your long-polling server is happy to keep the connection open for 60 (15-30s is a pretty common polling interval that minimizes the fuss here).

Server-Sent Events (SSE): The Efficient One-Way Street

SSE is an extension on long-polling that allows the server to send a stream of data to the client.

Normally HTTP responses have a header like `Content-Length` which tells the client how much data to expect. SSE instead uses a special header `Transfer-Encoding: chunked` which tells the client that the response is a series of chunks - we don't know how many there are or how big they are until we send them. This allows us to move from a single, atomic request/response to a more granular "stream" of data.

With SSE, instead of sending a full response once data becomes available, the server sends a chunk of data and then keeps the request open to send more data as needed. SSE is perfect for scenarios where servers need to push data to clients, but clients don't need to send data back frequently.

In our chat app, we would open up a request to stream messages and then each new message would be sent as a chunk to the client.

How SSE Works

1. Client establishes SSE connection
2. Server keeps connection open
3. Server sends messages when data changes or updates happen
4. Client receives updates in real-time

Modern browsers have built-in support for SSE through the `EventSource` object, making the client-side implementation straightforward.

```
// Client-side
const eventSource = new EventSource('/api/updates');

eventSource.onmessage = (event) => {
  const data = JSON.parse(event.data);
  updateUI(data);
};

// Server-side (Node.js/Express example)
app.get('/api/updates', (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');

  const sendUpdate = (data) => {
    res.write(`data: ${JSON.stringify(data)}\n\n`);
  };

  // Send updates when data changes
  dataSource.on('update', sendUpdate);
```

```
  // Clean up on client disconnect
  req.on('close', () => {
    dataSource.off('update', sendUpdate);
  });
});
```

Advantages

- Built into browsers.
- Automatic reconnection.
- Works over HTTP.
- More efficient than long polling due to less connection initiation/teardown.
- Simple to implement.

Disadvantages

- One-way communication only.
- Limited browser support (not an issue for modern browsers).
- Some proxies and networking equipment don't support streaming. Nasty to debug!
- Browsers limit the number of concurrent connections per domain, meaning you may only be able to have a few SSE connections per domain.
- Makes monitoring more painful since requests can hang around for a long time.

When to Use SSE

SSE is a great upgrade to long-polling because it eliminates the issues around high-frequency updates while still building on top of standard HTTP. That said, it comes with lesser overall support because you'll need both browsers and and/all infra between the client and server to support streaming responses.

A very popular use-case for SSE today is AI chat apps which frequently involve the need to stream new tokens (words) to the user as they are generated to keep the UI responsive.

An example of an infra gap is that many proxies and load balancers don't support streaming responses. In these cases, the proxy will try to buffer the response until it completes - which effectively blocks our stream in an annoying, opaque way that is hard to debug! [2]

As an aside: most interviewers will not be familiar with the **infrastructure considerations associated with SSE** and aren't going to ask you detailed questions about them. But if the role you're interviewing for is very frontend-centric be prepared in case they expect you to know your stuff!

Things to Discuss in Your Interview

SSE rides on existing HTTP infrastructure, so there's not a lot of extra infrastructure you'll need to talk through. You also don't have a polling interval to negotiate or tune.

Most SSE connections won't be super-long-lived (e.g. 30-60s is pretty typical), so if you need to send messages for a longer period you'll need to talk about how clients re-establish connections and how they deal with the gaps in between. The **SSE standard** includes a "last event ID" which is intended to cover this gap, and the `EventSource` object in browsers explicitly handles this reconnection logic. If a client loses its connection, it can reconnect and provide the last event ID it received. The server can then use that ID to send all the events that occurred while the client was disconnected.

Websockets: The Full-Duplex Champion

WebSockets are the go-to choice for true bi-directional communication between client and server. If you have high frequency writes *and* reads, WebSockets are the champ.

How WebSockets Works

Websockets build on HTTP through an "upgrade" protocol, which allows an existing TCP connection to change L7 protocols. This is super convenient because it means you can utilize some of the existing HTTP session information (e.g. cookies, headers, etc.) to your advantage.

> ⚠
> Just because clients can upgrade from HTTP to WebSocket doesn't mean that the infrastructure will support it. Every piece of infrastructure between the client and server will need to support WebSocket connections.

Once a connection is established, both client and server can send "messages" to each other which are effectively opaque binary blobs. You can shove strings, JSON, Protobufs, or anything else in there. Think of WebSockets like a TCP connection with some niceties that make establishing the connection easier, especially for browsers.

1. Client initiates WebSocket handshake over HTTP
2. Connection upgrades to WebSocket protocol
3. Both client and server can send messages
4. Connection stays open until explicitly closed

For our chat app, we'd connect to a WebSocket endpoint over HTTP, sharing our authentication token via cookies. The connection would get upgraded to a WebSocket connection and then we'd be able to receive messages back to the client over the connection as they happen. Bonus: we'd also be able to send messages to other users in the chat room!

```
// Client-side
const ws = new WebSocket('ws://api.example.com/socket');

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  handleUpdate(data);
};

ws.onclose = () => {
  // Implement reconnection logic
  setTimeout(connectWebSocket, 1000);
};

// Server-side (Node.js/ws example)
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  ws.on('message', (message) => {
    // Handle incoming messages
    const data = JSON.parse(message);
    processMessage(data);
  });

  // Send updates to client
  dataSource.on('update', (data) => {
    ws.send(JSON.stringify(data));
  });
});
```

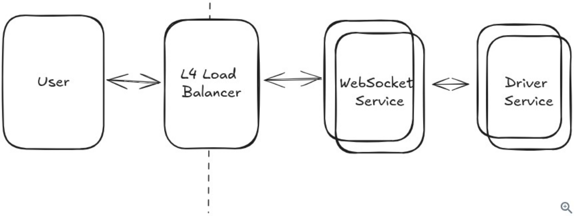
Extra Challenges

Because the Websocket is a *persistent connection*, we need our infrastructure to support it. Some L7 load balancers support websockets, but support is generally spotty (remember that L7 load balancers aren't guaranteeing we're using the same TCP connection for each incoming request). L4 load balancers will support websockets natively since the same TCP connection is used for each request.

When we have long-running connections we have another problem: deployments. When servers are redeployed, we either need to sever all old connections and have them reconnect or have the new servers take over and keep the connections alive. Generally speaking you should prefer the former since it's simpler, but it does have some ramifications on how "persistent" you expect the connection to be. You also need to be able to handle situations where a client needs to reconnect and may have missed updates while they were disconnected.

Finally, balancing load across websocket servers can be more complex. If the connections are truly long-running, we have to "stick with" each allocation decision we made. If we have a load balancer that wants to send a new request to a different server, it can't do that if it would break an existing websocket connection.

Because of all the issues associated with *stateful* connections, many architectures will terminate WebSockets into a WebSocket service early in their infrastructure. This service can then handle the connection management and scaling concerns and allows the rest of the system to remain *stateless*. The WebSocket service is also less likely to change meaning it requires less deployments which churn connections.



WebSocket Reference Architecture

Advantages

- Full-duplex (read and write) communication.
- Lower latency than HTTP due to reduced overhead (e.g. no headers).
- Efficient for frequent messages.
- Wide browser support.

Disadvantages

- More complex to implement.
- Requires special infrastructure.
- Stateful connections, can make load balancing and scaling more complex.
- Need to handle reconnection.

When to Use WebSockets

Generally speaking, if you need **high-frequency**, bi-directional communication, you're going to want to use WebSocket. I'm emphasizing high-frequency here because you can always make additional requests/connections for writes: a very common pattern is to have SSE subscriptions for updates and do writes over simple HTTP POST/PUT whenever they occur.

I often find candidates too eagerly adopting Websockets when they could be using SSE or simple polling instead. Because of the additional complexity and infra lift, you'll want to defer to SSE unless you have a specific need for this bi-directional communication.

Things to Discuss in Your Interview

Websockets are a powerful tool, but they do come with a lot of complexity. You'll want to talk through how you'll manage connections and deal with reconnections. You'll also need to consider how your deployment strategy will handle server restarts.

Managing *statefulness* is a big part of the conversation. Senior/staff candidates will frequently talk about how to minimize the spread of state across their architecture.

There's also a lot to discuss about how to scale WebSocket servers. Load can be uneven which can result in hotspots and failures. Using a "least connections" strategy for the load balancer can help, as well as minimizing the amount of work the WebSocket servers need to do as they process messages. Using the reference architecture above and offloading more intensive processing to other services (which can scale independently) can help.

WebRTC: The Peer-To-Peer Solution

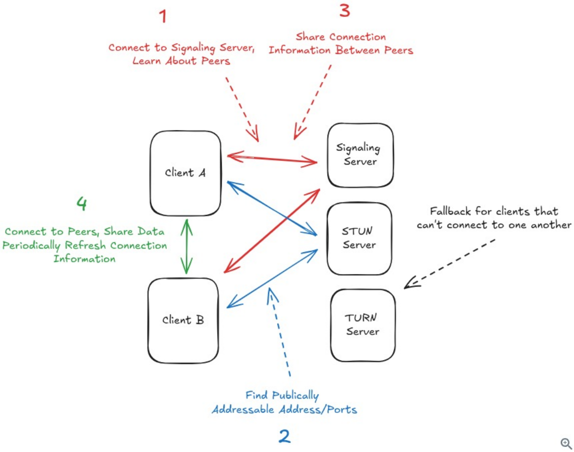
Our last option is the most unique. WebRTC enables direct peer-to-peer communication between browsers, perfect for video/audio calls and some data sharing like document editors.

Clients talk to a central "signaling server" which keeps track of which peers are available together with their connection information. Once a client has the connection information for another peer, they can try to establish a direct connection without going through any intermediary servers.

In practice, most clients don't allow inbound connections for security reasons (the exception would be servers which broadcast their availability on specific ports at specific addresses) using devices like NAT (network address translation). So if we stopped there, most peers wouldn't be able to "speak" to each other.

The WebRTC standard includes two methods to work around these restrictions:

- **STUN**: "Session Traversal Utilities for NAT" is a protocol and a set of techniques like "hole punching" which allows peers to establish publically routable addresses and ports. I won't go into details here, but as hacky as it sounds it's a standard way to deal with NAT traversal and it involves repeatedly creating open ports and sharing them via the signaling server with peers.
- **TURN**: "Traversal Using Relays around NAT" is effectively a relay service, a way to bounce requests through a central server which can then be routed to the appropriate peer.


1
Connect to Signaling Server, Learn About Peers
3
Share Connection Information Between Peers
Client A
Signaling Server
4
Connect to Peers, Share Data
Periodically Refresh Connection Information
STUN Server
Fallback for clients that can't connect to one another
Client B
TURN Server
Find Publically Addressable Address/Ports
2

WebRTC Setup

In practice, the signaling server is relatively lightweight and isn't handling much of the bandwidth as the bulk of the traffic is handled by the peer-to-peer connections. But interestingly the signaling server does effectively act as a real-time update system for its clients (so they can find their peers and update their connection info) so it either needs to utilize WebSockets, SSE, or some other approach detailed above.

For our chat app, we'd connect to our signaling server over a WebSocket connection to find all of our peers (others in the chat room). Once we'd identified them and exchanged connection information, we'd be able to establish direct peer-to-peer connections with them. Chat messages would be broadcast by room participants to all of their peers (or, if you want to be extra fancy, bounced between participants until they settle).

How WebRTC Works

Ok, but how does it work?

1. Peers discover each other through signaling server.
2. Exchange connection info (ICE candidates)
3. Establish direct peer connection, using STUN/TURN if needed
4. Stream audio/video or send data directly

Pretty simple, apart from the acronyms and NAT traversal.

```
// Simplified WebRTC setup
async function startCall() {
  const pc = new RTCPeerConnection();

  // Get local stream
  const stream = await navigator.mediaDevices.getUserMedia({
    video: true,
    audio: true
  });
```

```
  // Add tracks to peer connection
  stream.getTracks().forEach(track => {
    pc.addTrack(track, stream);
  });

  // Create and send offer
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);

  // Send offer to signaling server
  signalingServer.send(offer);
}
```

When to Use WebRTC

WebRTC is the most complex and heavyweight of the options we've discussed. It's overkill for most real-time update use cases, but it's a great tool for scenarios like video/audio calls, screen sharing, and gaming.

The notable exception is that it can be used to reduce server load. If you have a system where clients need to talk to each other frequently, you could use WebRTC to reduce the load on *your* servers by having clients establish their own connections. **Canva took this approach with presence/pointer sharing in their canvas editor** and it's a popular approach from collaborative document editing like **Google Docs** when used in conjunction with CRDTs which are better suited for a peer-to-peer architecture.

Advantages

- Direct peer communication
- Lower latency
- Reduced server costs
- Native audio/video support

Disadvantages

- Complex setup (> WebSockets)
- Requires signaling server
- NAT/firewall issues
- Connection setup delay

Things to Discuss in Your Interview

If you're building a WebRTC app in a system design interview, it should be really obvious why you're using it. Either you're trying to do video conferencing or the scale strictly requires you to have clients talk to each other directly.

> Some interviews will get cute and introduce unrealistic constraints to try to get you to think outside the box, like "the system must run on Raspberry Pi". These *might* be a case where a peer-to-peer architecture makes sense, but tread carefully.

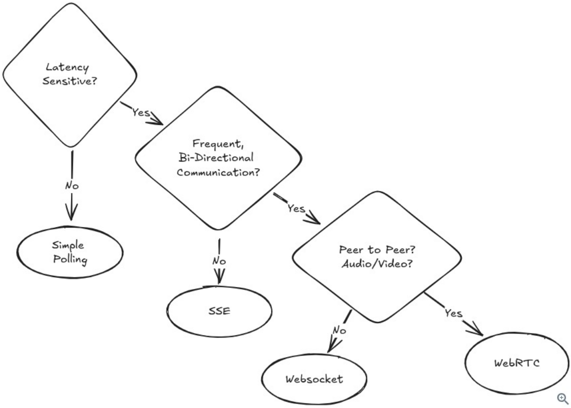
Having some knowledge of the infra requirements (STUN/TURN, signaling servers, etc.) will give you the flexibility to make the best design decision for your system. You'll also want to speak pretty extensively about the communication patterns between peer clients and any eventual synchronization to a central server (almost all design questions will have some calling home to the mothership to store data or report results).

Overview

There are a lot of options for delivering events from the server to the client. Being familiar with the trade-offs associated with each will give you the flexibility to make the best design decision for your system. If you're in a hurry, the following flowchart will help you choose the right tool for the job.

- If you're not latency sensitive, **simple polling** is a great baseline. You should probably start here unless you have a specific need in your system.
- If you don't need bi-directional communication, **SSE** is a great choice. It's lightweight and works well for many use cases. There are some infrastructure considerations to keep in mind, but they're less invasive than with WebSocket and generally interviewers are less familiar with them or less critical if you don't address them.
- If you need frequent, bi-directional communication, **WebSocket** is the way to go. It's more complex, but the performance benefits are huge.
- Finally, if you need to do audio/video calls, **WebRTC** is the only way to go. In some instances peer-to-peer

collaboration can be enhanced with WebRTC, but you're unlikely to see it in a system design interview.

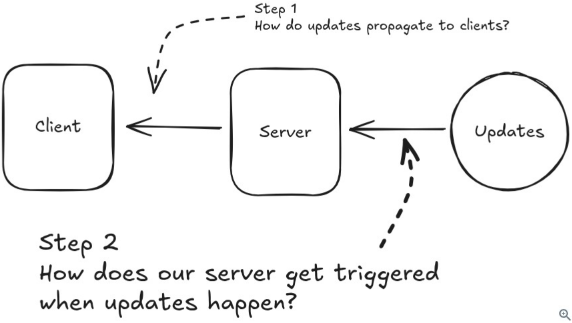

Latency Sensitive?
Yes
No
Simple Polling
Frequent, Bi-Directional Communication?
Yes
No
SSE
Peer to Peer? Audio/Video?
Yes
No
Websocket
WebRTC

Client Updates Flowchart

But now that we have the first hop out of the way, let's talk about how updates propagate from their source to the server in question.

Server-Side Push/pull

Now that we've established our options for the hop from server to client (Simple Polling, Long-Polling, SSE, WebSockets, WebRTC), let's talk about how we can propagate updates from the source to the server.


Step 1
How do updates propagate to clients?
Client
Server
Updates
Step 2
How does our server get triggered when updates happen?

Server-Side Push/Pull

Invariably our system is somehow *producing* updates that we want to propagate to our clients. This could be other users making edits to a shared documents, drivers making location updates, or friends sending messages to a shared chat.

Making sure these updates get to their ultimate destination is closely tied to how we propagate updates from the source to the server. Said differently, we need a **trigger**.

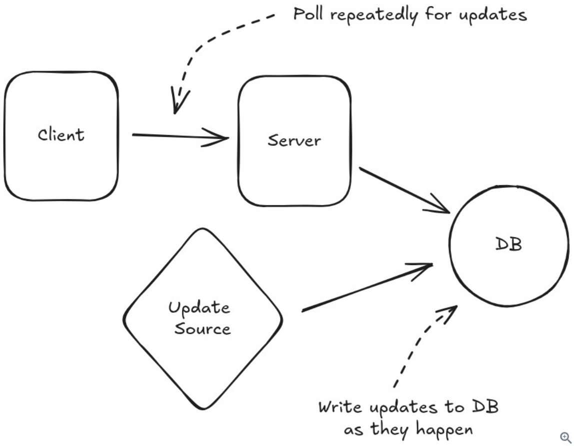
When it comes to triggering, there's three patterns that you'll commonly see:

1. Pulling via Polling
2. Pushing via Consistent Hashes

3. Pushing via Pub/Sub

Pulling With Simple Polling

With Simple Polling, we're using a **pull-based** model. Our client is constantly asking the server for updates and the server needs to maintain the state necessary to respond to those requests. The most common way to do this is to have a database where we can store the updates (e.g. all of the messages in the chat room), and from this database our clients can pull the updates they need when they can. For our chat app, we'd basically be polling for "what messages have been sent to the room with a timestamp larger than the last message I received?".


Poll repeatedly for updates
Client
Server
DB
Update Source
Write updates to DB as they happen

Pulling with Simple Polling

Remember with polling we're tolerating delay! We use the poll itself as the trigger, even though the actual update may have occurred some time prior.

The nice thing about this from a system design perspective is that we've *decoupled* the source of the update from the client receiving it. The line that receives the updates is interrupted (by the DB) from the line that produces them - data is not *flowing* from the producer to the consumer. The downside is that we've lost the real-time aspect of our updates.

Advantages

- Dead simple to implement.
- State is constrained to our DB.
- No special infrastructure.
- Doesn't take much time to explain.

Disadvantages

- High latency.
- Excess DB load when updates are infrequent and polling is frequent.

When to Use Pull-Based Polling

Pull-based polling is great when you want your user experience to be somewhat more responsive to changes that happen on the backend, but responding quickly is not the main thing. Generally speaking, if you need real-time updates this is not the best approach, but again there are a lot of use-cases where real-time updates are actually not required!

Things to Discuss in Your Interview

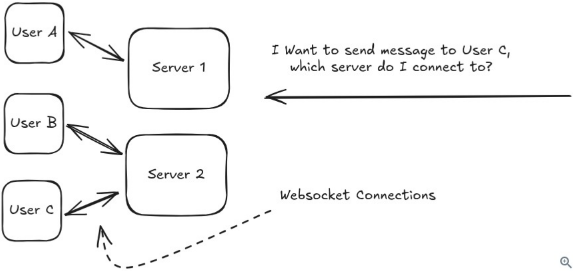
When you're using Pull-based Polling, you'll want to talk about how you're storing the updates. If you're using a database, you'll want to discuss how you're querying for the updates and how that might change given your load.

In many instances where this approach might be used, the number of clients can actually be quite large. If you have a million clients polling every 10 seconds, you've got 100k TPS of read volume! This is easy to forget about.

Pushing Via Consistent Hashes

The remaining approaches involve **pushing** updates to the clients. In many of the client update mechanisms that we discussed above (long-polling, SSE, WebSockets) the client has a persistent connection to one server and that server is responsible for sending updates to the client.

But this creates a problem! For our chat application, in order to send a message to `User C`, we need to know which server they are connected to.


User A
Server 1
User B
Server 2
User C
I Want to send message to User C, which server do I connect to?
Websocket Connections

Push Problems

Ideally, when an a message needs to be sent, I would:

1. Figure out which server `User C` is connected to.
2. Send the message to that server.
3. That server will look up which (websocket, SSE, long-polling) request is associated with `User C`.
4. The server will then write the message via the appropriate connection.

There are two common ways to handle this situation, and the first is to use **hashing**. Let's build up our intuition for this in two steps.

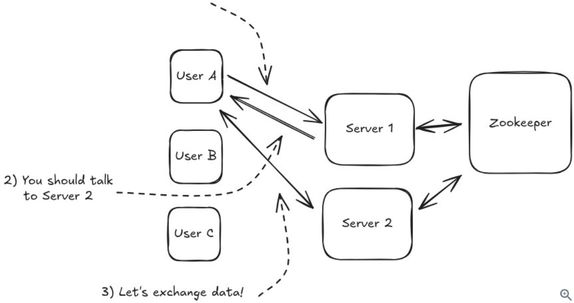
Simple Hashing

Our first approach might be to use a simple modulo operation to figure out which server is responsible for a given user. Then, we'll always have 1 server who "owns" the connections for that user.

To do this, we'll have a central service that knows how many servers there are `N` and can assign them each a number `0` through `N-1`. This is frequently Apache **ZooKeeper** or Etcd which allows us to manage this metadata and allows the servers to keep in sync as it updates, though in practice there are many alternatives.

We'll make the server number `n` responsible for user `u % N`. When clients initially connect to our service, we can either:
a. Directly connect them to the appropriate server (e.g. by having them know the hash, `N`, and the server addressess associated with each index).
b. Have them randomly connect to any of the servers and have that server redirect them to the appropriate server based on internal data.

1) I'm trying to connect for User A



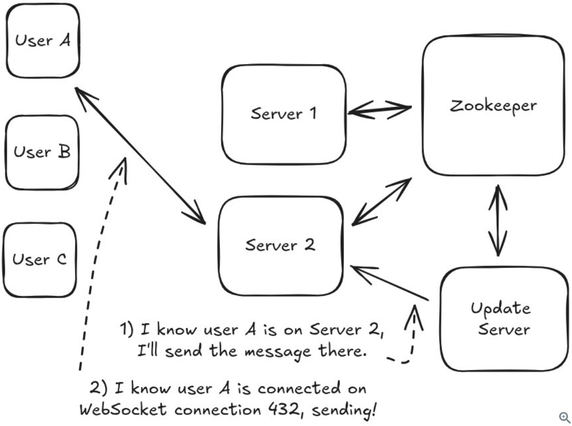
Connecting to the Right Server

When a client connects, the following happens:

1. The client connects to a random server.
2. The server looks up the client's hash in Zookeeper to figure out which server is responsible for them.
3. The server redirects the client to the appropriate server.
4. The client connects to the correct server.
5. The server adds that client to a map of connections.

Now we're ready to send updates and messages!

When we want to send messages to `User C`, we can simply hash the user's id to figure out which server is responsible for them and send the message there.


User A
User B
User C
Server 1
Server 2
Zookeeper
Update Server
1) I know user A is on Server 2, I'll send the message there.
2) I know user A is connected on WebSocket connection 432, sending!

Sending Updates to the Right Server

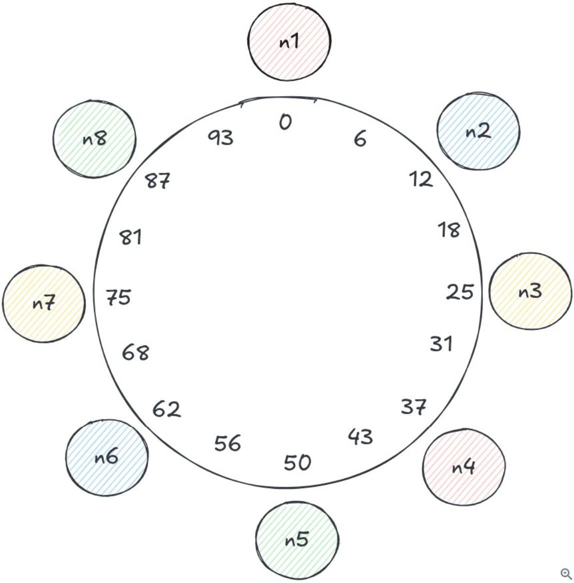
1. Our Update Server stays connected to Zookeeper and knows the addresses of all servers and the modulo `N`.
2. When the Update Server needs to send a message to `User C`, it can hash the user's id to figure out which server is responsible for them (Server 2) and sends the message there.
3. Server 2 receives the message, looks up which connection is associated with `User C`, and sends the message to that connection.

This approach works because we always know that a single server is responsible for a given user (or entity, or ID, or whatever). All inbound connections go to that server and, if we want to use the connection associated with that entity, we know to pass it to that server for forwarding to the end client.

Consistent Hashing

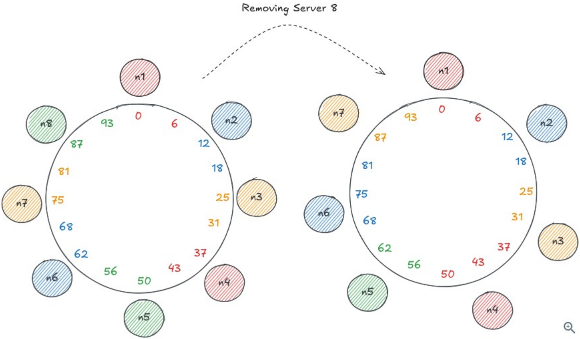
The hashing approach works great when `N` is fixed, but becomes problematic when we need to scale our service up or down. With simple modulo hashing, changing the number of servers would require almost all users to disconnect and reconnect to different servers - an expensive operation that disrupts service.

Consistent hashing solves this by minimizing the number of connections that need to move when scaling. It maps both servers and users onto a hash ring, and each user connects to the next server they encounter when moving clockwise around the ring.



Consistent Hash Ring

When we add or remove servers, only the users in the affected segments of the ring need to move. This greatly reduces connection churn during scaling operations.



Advantages

- Predictable server assignment
- Minimal connection disruption during scaling
- Works well with stateful connections
- Easy to add/remove servers

Disadvantages

- Complex to implement correctly
- Requires coordination service (like Zookeeper)
- All servers need to maintain routing information
- Connection state is lost if a server fails

When to Use Consistent Hashing

Consistent hashing is ideal when you need to maintain persistent connections (WebSocket/SSE) and your system needs to scale dynamically. It's particularly valuable when each connection requires significant server-side state that would be expensive to transfer between servers.

For example, in **the Google Docs design**, connections are associated with specific documents that require substantial state to maintain collaborative editing functionality. Consistent hashing helps keep that state on a single server while allowing for scaling.

However, if you're just passing small messages to clients without much associated state, you're probably better off using the next approach: Pub/Sub.

Things to Discuss in Your Interview

If you introduce a consistent hashing approach in an interview, you'll want to be able to discuss not only how the updates are routed (e.g. a cooordination service like Zookeeper or etcd). Interviewers are usually interested to understand how the system scales: what happens when we need to increase or decrease the number of nodes. For those instances, you'll want to be able to share your knowledge about consistent hashing but also talk about the orchestration logic necessary to make it work. In practice, this usually means:

1. Signaling the beginning of a scaling event. Recording both the old and new server assignments.
2. Slowly disconnecting clients from the old server and having them reconnect to their newly assigned server.
3. Signaling the end of the scaling event and updating the coordination service with the new server assignments.
4. In the interim, having messages which are sent to both the old and new server until they're fully transitioned.

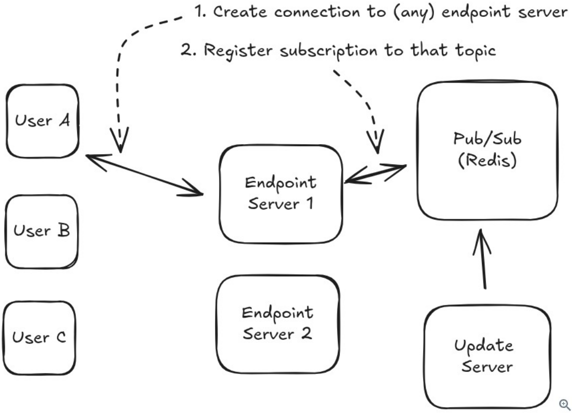
The mechanics of discovering the initial server assignments is also interesting. Having clients know about the internal structure of your system can be problematic, but there are performance tradeoffs associated with redirecting clients to the correct server or requiring them to do a round-trip to a central server to look up the correct one. Especially during scaling events, any central registration service may become a bottleneck so it's important to discuss the tradeoffs with your interviewer.

Pushing Via Pub/Sub

Another approach to triggering updates is to use a **pub/sub** model. In this model, we have a single service that is responsible for collecting updates from the source and then broadcasting them to all interested clients. Popular options here include Kafka and Redis.

The pub/sub service becomes the biggest source of *state* for our realtime updates. Our persistent connections are now made to lightweight servers which simply subscribe to the relevant topics and forward the updates to the appropriate clients. I'll refer to these servers as *endpoint* servers.

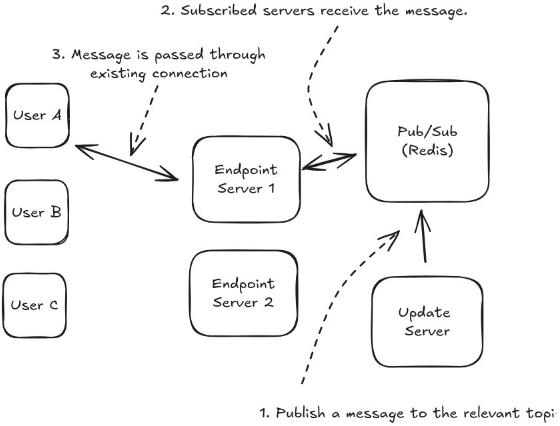
When clients connect, we don't need them to connect to a specific endpoint server (like we did with consistent hashing) and instead can connect to any of them. Once connected, the endpoint server will register the client with the pub/sub server so that any updates can be sent to them.


1. Create connection to (any) endpoint server
2. Register subscription to that topic
User A
User B
User C
Endpoint Server 1
Endpoint Server 2
Pub/Sub (Redis)
Update Server

Pub/Sub

On the connection side, the following happens:

1. The client establishes a connection with an endpoint server.
2. The endpoint server registers the client with the Pub/Sub service, often by creating a topic, subscribing to it, and keeping a mapping from topics to the connections associated with them.


2. Subscribed servers receive the message.
3. Message is passed through existing connection
User A
User B
User C
Endpoint Server 1
Endpoint Server 2
Pub/Sub (Redis)
Update Server
1. Publish a message to the relevant topi

Pub/Sub Message Sending

On the update broadcasting side, the following happens:

1. Updates are pushed to the Pub/Sub service to the relevant topic.
2. The Pub/Sub service broadcasts the update to all clients subscribed to that topic.
3. The endpoint server receives the update, looks up which client is subscribed to that topic, and forwards the update to that client over the existing connection.

For our chat application, we'll create a topic for each user. When the client connects to our endpoint, it will subscribe to the topic associated with the connected user. When we need to send messages, we publish them to that user's topic and the Pub/Sub service will broadcast them to all of the subscribed servers - then these servers will forward them to the appropriate clients over the existing connections.

Advantages

- Managing load on endpoint servers is easy, we can use a simple load balancer with "least connections" strategy.
- We can broadcast updates to a large number of clients efficiently.
- We minimize the proliferation of state through our system.

Disadvantages

- We don't know whether subscribers are connected to the endpoint server, or when they disconnect.
- The Pub/Sub service becomes a single point of failure and bottleneck.
- We introduce an additional layer of indirection which can add to latency.
- There exist many-to-many connections between Pub/Sub service hosts and the endpoint servers.

When to Use Pub/Sub

Pub/Sub is a great choice when you need to broadcast updates to a large number of clients. It's easy to set up and requires little overhead on the part of the endpoint servers. The latency impact is minimal (<10ms). If you don't need to respond to connect/disconnect events or maintain a lot of state associated with a given client, this is a great approach.

Things to Discuss in Your Interview

If you're using a pub/sub model, you'll probably need to talk about the single point of failure and bottleneck of the pub/sub service. Redis cluster is a popular way to scale pub/sub service which involves sharding the subscriptions by their key across multiple hosts. This scales up the number of subscriptions you can support and the throughput.

Introducing a cluster for the Pub/Sub component means you'll manage the many-to-many connections between the pub/sub service and the endpoint servers (each endpoint server will be connected to all hosts in the cluster). In some cases this can be managed by carefully choosing topics to partition the service, but in many cases the number of nodes in the cluster is small.

For inbound connections to the endpoint servers, you'll probably want to use a load balancer with a "least connections" strategy. This will help ensure that you're distributing the load across the servers in the cluster. Since the connection itself (and the messages sent across it) are effectively the only resource being consumed, load balancing based on connections is a great way to manage the load.

When to Use in Interviews

Real-time updates appear in almost every system design interview that involves user interaction or live data. Rather than waiting for the interviewer to ask about real-time features, proactively identify where immediate updates matter and address them in your initial design.

A strong candidate recognizes real-time requirements early. When designing a chat application, immediately acknowledge that "messages need to be delivered instantly to all participants - I'll address that with WebSockets." For collaborative editing, mention that "character-level changes need sub-second propagation between users."

Common Interview Scenarios

Chat Applications - The classic real-time use case. Messages must appear instantly across all participants. WebSockets handle the bidirectional communication perfectly, while pub/sub distributes messages to the right servers. Consider message ordering, typing indicators, and presence status.

Live Comments - High-volume, real-time social interaction during live events. Millions of viewers commenting simultaneously creates extreme fan-out problems. Hierarchical aggregation and careful batching prevent system overload while maintaining the live feel.

Collaborative Document Editing - Character-level changes need instant propagation between users. WebSockets provide the low-latency communication, while operational transforms or CRDTs handle conflict resolution. User cursors and selection highlighting add additional real-time complexity.

> Collaborative editing commonly requires us to not only deal with getting updates to clients with low latency and high frequency, but also to deal with conflicts and ensure that the state of the document is consistent. If users can be typing on top of one another, they often need to be able to deal with the conflicts that arise. We talk about two approaches for dealing with this in the Google Docs breakdown: CRDTs and Operational Transforms.

Live Dashboards and Analytics - Business metrics and operational data that changes constantly. Server-Sent Events work well for one-way data flow from servers to dashboards. Consider data aggregation intervals and what constitutes "real-time enough" for business decisions.

Gaming and Interactive Applications - Multiplayer games need the lowest latency possible. WebRTC enables peer-to-peer communication for reduced latency, while WebSockets handle server coordination. Consider different update frequencies for different game elements.

When NOT to Use

Avoid real-time updates when you can get away with a simple polling model. If you're not latency sensitive, polling is a great baseline and minimizes complexity — a property highly valued in senior+ interviews. By polling you avoid both hops: you don't need to worry about the client->server protocols AND you don't have to handle propagation from the event source.

Common Deep Dives

Interviewers love to probe the operational challenges and edge cases of real-time systems. Here are the most common follow-up questions you'll encounter.

"How do you handle connection failures and reconnection?"

Real-world networks are unreliable. Mobile users lose connections constantly, WiFi drops out, and servers restart. Your real-time system needs graceful degradation and recovery.

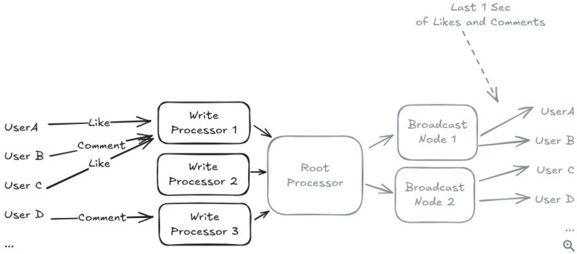
The key challenge is detecting disconnections quickly and resuming without data loss. WebSocket connections don't always signal when they break - a client might think it's connected while the server has already cleaned up the connection. Implementing heartbeat mechanisms helps detect these "zombie" connections.

For recovery, you need to track what messages or updates a client has received. When they reconnect, they should get everything they missed. This often means maintaining a per-user message queue or implementing sequence numbers that clients can reference during reconnection. Using **Redis** streams for this is a popular option.

"What happens when a single user has millions of followers who all need the same update?"

This is the classic "celebrity problem" in real-time systems. When a celebrity posts, millions of users need that update simultaneously. Naive approaches create massive fan-out that can crash your system.

The solution involves strategic caching and hierarchical distribution. Instead of writing the update to millions of individual user feeds, cache the update once and distribute through multiple layers. Regional servers can pull the update and push to their local clients, reducing the load on any single component. More details on this in the **Batching and Hierarchical Aggregation** section of the Scaling Writes pattern.



"How do you maintain message ordering across distributed servers?"

When multiple servers handle real-time updates, ensuring consistent ordering becomes complex. Two messages sent milliseconds apart might arrive out of order if they travel different network paths or get processed by different servers.

Vector clocks or logical timestamps help establish ordering relationships between messages. Each server maintains its own clock, and messages include timestamp information that helps recipients determine the correct order.

For critical ordering requirements, you might need to funnel all related messages through a single server or partition. This trades some scalability for consistency guarantees, but simplifies the ordering problem significantly.

> For most *product*-style system design interviews, using a single server or partition is the way to go. There's a place for vector clocks and other techniques but they most often apply to deep infra rather than a question like "Design an Online Auction System". If all your messages make their way to a single host, stamping them with the correct timestamp and establishing a total order is straightforward.

Conclusion

Real-time updates are among the most challenging patterns in system design, appearing in virtually every interactive application from messaging to collaborative editing. The key insight is that real-time systems require solving two distinct problems: client-server communication protocols and server-side update propagation.

Start simple and escalate based on requirements. If polling every few seconds meets your needs, don't jump to complex WebSocket architectures. Most candidates over-engineer real-time solutions when simpler approaches would suffice. However, when true real-time performance is required, understanding the trade-offs between protocols becomes crucial.

For client communication, SSE and WebSockets handle most real-time scenarios effectively. SSE works brilliantly for one-way updates like live dashboards, while WebSockets excel when you need bidirectional communication. Both are well-supported and understood by most infrastructure teams.

On the server side, pub/sub systems provide the best balance of simplicity and scalability for most applications. They decouple update sources from client connections, making your system easier to reason about and scale. Reserve consistent hashing approaches for scenarios where connection state management becomes a primary concern.

Overall real-time update applications bring a lot of interesting complexity to system design: low latency, scaling, networking issues, and the need to manage multiple services and state. By learning about the options available to you, you'll be able to make the best design decision for your system and communicate your reasoning to your interviewer. Good luck!

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned.

Start Quiz

1. DNS technically can run over TCP or UDP, but we'll exclude that for simplicity in this illustration. ↩

2. We use SSE extensively for Hello Interview and the amount of time we've spent dealing with networking edge cases is mind boggling. ↩

Mark as read ○

Add a comment...

Comment

○ Anonymous

Posting as leetcode

Sort By

Search comments

Old

TrickyTanIguana957 • 9 months ago

This is an awesome deep-dive!

69 Reply

> **Stefan Mai** • 8 months ago
>
> Thanks! Glad you like it.
>
> 9 Reply

sahil chug • 8 months ago

This is just amazing!!

1 Reply

manivannan.sivaraj • 8 months ago

Wonderful article! Thanks for putting it all together in one :)

4 Reply

NetBlackMite212 • 8 months ago

Great deep dive! Qq: what are some common strategies to discuss pub/sub single point of failures?

3 Reply

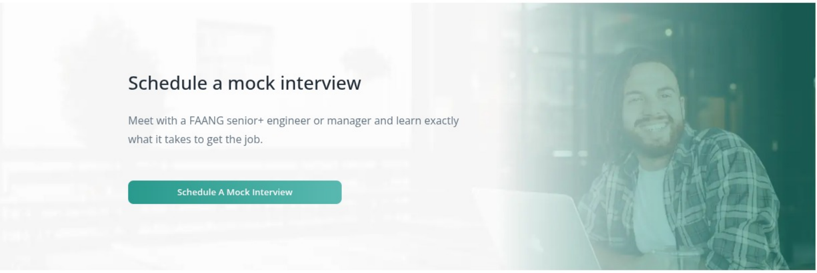
> **NetBlackMite212** • 8 months ago
>
> And also if we use redis pub/sub, what are some common ways to handle messages being dropped with redis' at most one message sending
>
> 5 Reply

CurlyAmberAlpaca598 • 8 months ago

How to scale websockets & SSE to handle millions of connections simulateneously?

6 Reply

Show All Comments



Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

Schedule A Mock Interview

Questions

- Meta SWE Interview Questions
- Amazon SWE Interview Questions
- Google SWE Interview Questions
- Engineering Manager (EM) Interview Questions

Learn

- Learn System Design
- Learn DSA
- Learn Behavioral
- Learn ML System Design
- Guided Practice

Links

- FAQ
- Pricing
- Gift Mock Interviews
- Gift Premium
- Become a Coach
- Our Coaches
- Employers
- Hello Interview Premium

Legal

- Terms and Conditions
- Privacy Policy

Contact

About Us

support@hellointerview.com

7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103

© 2025 Optick Labs Inc. All rights reserved.